Les tâches d'Incognita

Chaque tâche, ses critères, et la comparaison avec LMLExplore

Adrien Prevost
rédigé avec l'aide de Claude (Anthropic)

13 novembre 2026 — état du dépôt après les PR #1 à #4

À retenir

Ce document décrit ce que font les scripts et les consignes *tels qu'ils sont écrits* (`AGENTS.md`, `agents/roles/*.md`, `scripts/agent.sh`, `scripts/loop.sh`, la CI). Les scripts d'agents ont tourné pour la première fois le 13/11 au soir (essai du bibliographe) ; le reste n'a pas encore été observé en conditions réelles.

Table des matières

1 Vue d'ensemble **1**

2 Les tâches des agents **2**

3 Tes tâches et celles des automates **6**

4 Comparaison avec LMLExplore **6**

4.1 Ce que fait le worker Tau Ceti 6

4.2 Tableau comparatif 7

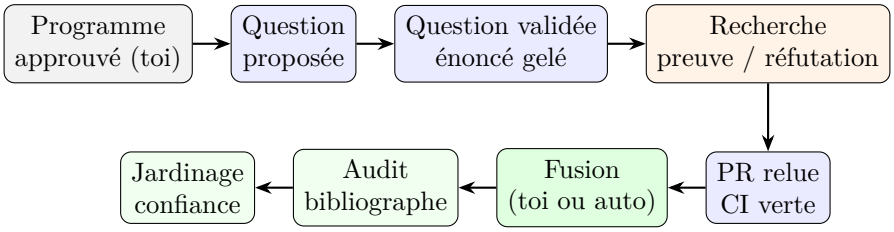
5 Premier essai réel (13 novembre 2026) **7**

1 Vue d'ensemble

Il y a trois sortes d'acteurs :

- **toi** (les humains) : écrire et approuver les programmes, décider pendant la calibration, fusionner, auditer par échantillon ;
- **les agents**, une session Claude par tâche, lancée par `scripts/agent.sh` ;
- **les automates** : la boucle `scripts/loop.sh` (choix des tâches, quota, fusion automatique), la CI de GitHub (compilation, audit, PDF).

Le cycle d'un résultat



Les résultats *connus* dont un programme a besoin suivent un chemin plus court : formalisation directe (énoncé gelé tiré de la source), relue pour sa fidélité à la source, sans question.

Ordre de priorité de la boucle

À chaque tour, `scripts/loop.sh` : (0) fusionne ce qui peut l'être automatiquement, (0') attend que le quota le permette, puis lance *une* tâche, la première disponible dans cet ordre :

	Tâche	Quand
1	relire une PR	une PR ouverte n'a pas d'avis `Verdict:` sur son dernier commit
2	jardiner	un tour sur dix
3	valider une question	une question `proposed` n'a pas encore d'avis du relecteur
4	bibliographe (audit)	un résultat du catalogue a `novelty = "unchecked"`
5	bibliographe (routine)	un tour sur quatre
6	recherche	une question `open` existe, un tour sur deux
7	formaliser un résultat connu	deux tours sur trois parmi les restants
8	proposer une question	sinon

Les tâches 2 à 8 ne portent que sur les programmes **approuvés** (aujourd'hui : BatchedBandits).

2 Les tâches des agents

Chaque fiche donne : le *rôle* (fichier de consignes), le *déclencheur*, ce que l'agent *lit*, ce qu'il *écrit*, les *critères* qu'il doit appliquer, et les *contrôles* qui ne dépendent pas de lui (script, CI, relecteur). Tous les agents peuvent lire le web (recherche et pages), mais n'ont ni réseau en ligne de commande, ni `git push`, ni `gh` : c'est le script qui pousse la branche et ouvre la PR. Tous ceux qui écrivent terminent par un résumé (`.agents/summary.md`) qui devient la description de la PR et le PDF.

1. Proposer une question — rôle `propose` (modèle auteur)

Déclencheur. rien de plus prioritaire ; programme approuvé tiré au hasard.

Lit. le programme et ses fichiers compagnons, l'index et toutes les questions existantes, le catalogue.

Écrit. un fichier `programmes/<P>/questions/Q<nnn>-<titre>.md` (gabarit `templates/QUESTION.md`), statut `proposed`, « décision du propriétaire requise : oui », et une ligne dans l'index. Éventuellement un brouillon dans `Exploration/`.

Critères.

— entrer dans une direction ouverte du programme et satisfaire *chacun* de ses critères de validation (vérifiés par l'agent lui-même avant d'écrire) ;

— ne dupliquer aucune question existante, ne pas être un corollaire du catalogue (essayer d'abord une preuve courte ; si elle marche, choisir autre chose) ;

— chercher la réponse dans la littérature ; si elle est connue, changer de question ou la transformer en résultat connu à formaliser ;

— préférer ce qui compterait pour le programme à ce qui est seulement faisable ; une question dont la réponse probable est « non » est acceptable ;

— contenir un exemple qui satisfait toutes les hypothèses à la fois, et la variante de repli.

Contrôles. le script refuse tout fichier hors des questions et d'`Exploration/` ; la question passe ensuite par la tâche 2.

2. Valider une question — rôle `review-question` (modèle relecteur)

Déclencheur. une question `proposed` sans avis du relecteur (priorité haute).

Lit. uniquement la question, son programme et le catalogue ; pas l'historique git ni ce que le proposeur a écrit ailleurs (indépendance).

Écrit. son verdict (`accept` ou `reject`, modèle, date) et un rapport critère par critère dans la question ; s'il accepte, l'**énoncé gelé** en Lean (`Statements/<P>/Q<nnn>.lean`, avec `sorry`), vérifié par la compilation.

Critères.

— chaque critère de validation du programme : « rempli » ou « non rempli », avec une phrase ;

— tentative de preuve courte à partir du catalogue et des dépendances : si elle réussit, rejet (corollaire) ;

— recherche de la réponse dans la littérature : si elle est connue, rejet avec la référence ;

— intérêt pour le programme (section « ce qui serait un résultat important ») ;

— l'énoncé gelé n'utilise que des définitions de la bibliothèque et des dépendances.

Contrôles. pendant la **calibration**, le statut reste `proposed` : c'est toi qui décides, et l'accord entre ton avis et celui du relecteur est compté. L'ajout d'un fichier dans `Statements/` n'est jamais fusionné automatiquement.

3. Recherche sur une question ouverte — rôle `research` (modèle auteur)

Déclencheur. une question `open` existe (un tour sur deux).

Lit. la question et ses impasses, le programme, l'énoncé gelé, le catalogue.

Écrit. selon l'issue : brouillons dans `Exploration/` ; preuve dans la bibliothèque sous le nom `Incognita.<P>.<nom>` ; rédaction lisible avec rubrique « Attribution » ; entrée du catalogue avec `novelty = "unchecked"` ; statut `proved`, `refuted` ou `variant-proved` ; sinon une ligne d'impasse par approche ratée.

Critères.

— prouver *exactement* l'énoncé gelé, jamais l'affaiblir ;

— une réfutation (preuve de la négation) est un résultat à part entière ;

— une variante exige une raison formelle : contre-exemple à l'original, ou preuve que l'hypothèse ajoutée est nécessaire, ou étiquette « original toujours ouvert » ;

— bibliothèque : aucun `sorry`, axiomes standard seulement, aucun avertissement ;

— toute nouvelle définition : témoin non trivial, retour aux cas connus, non-vacuité

conjointe des hypothèses, finesse quand c'est possible ;

— ne jamais déclarer le résultat nouveau soi-même ; citer seulement des références lues.

Contrôles. CI : compilation sans `sorry` ni avertissement, `lake exe audit` (axiomes, et preuve du même type que l'énoncé gelé, testé : il rejette les deux fautes) ; relecture (tâche 6) ; audit du bibliographe après fusion.

4. Formaliser un résultat connu — rôle `formalise` (modèle auteur)

Déclencheur. un programme approuvé a une liste « Known results to formalise » (deux tours sur trois parmi les restants).

Lit. le programme, le catalogue, la fiche de lecture de la source.

Écrit. l'énoncé gelé avec la citation exacte `[CLÉ, Théorème X]` et les différences de forme ; la preuve dans la bibliothèque ; l'entrée du catalogue (`source`, `novelty = "known: ..."`) ; au besoin la fiche de lecture.

Critères.

— prendre le premier élément non fait, non marqué *on hold*, absent des dépendances ; s'il est trop gros, une partie autonome (une définition et son API, un lemme) ;

— formaliser à partir de la source, jamais de mémoire : l'énoncé lu doit être recopié dans `literature/notes/` ;

— suivre les noms et formes d'énoncés prévus par les feuilles de route de LMLExplore, pour qu'une adoption ultérieure soit une suppression plus un import ;

— si la source n'est pas publiée et que la preuve échoue à cause de l'énoncé : le signaler dans une question, ne jamais affaiblir en silence.

Contrôles. les mêmes que la tâche 3 ; la relecture vérifie la fidélité à la source plutôt que la nouveauté.

5. Bibliographe — rôle `bibliographer` (modèle auteur)

Déclencheur. un résultat non audité (priorité haute), sinon un tour sur quatre.

Fait. une seule de ces trois choses, dans cet ordre :

1. **audit d'attribution** d'un résultat fusionné : le résultat est-il connu, cas particulier d'un résultat connu ? d'où vient chaque technique de la preuve (la plus ancienne source, en suivant les références dans les deux sens) ? Il fixe `novelty` à `new`, `rediscovery of [...]` ou `known: [...]`, complète la rubrique « Attribution » et ajoute `attribution-audit` aux vérifications ;
2. **vérifier une référence** : titre, auteurs, revue, année, identifiant comparés à la page de l'éditeur ou d'arXiv ; correction de `bibliography.json`, passage à `checked` ;
3. **lire un passage cité** : recopier dans la fiche l'énoncé exact (hypothèses, conclusion, constantes), marquer le passage comme lu.

Critères. ne consigner que ce qui a été lu dans la source ; ne rien remplir de mémoire ; ne pas faire confiance aux rubriques « Attribution » des autres, les vérifier ; citations courtes seulement.

Contrôles. le script refuse toute écriture hors de `literature/`, `catalogue/` et des rédactions de questions.

6. Relire une PR — rôle `review-pr` (modèle relecteur)

Déclencheur. une PR ouverte sans avis sur son dernier commit (priorité la plus haute).

Lit. le diff et la description de la PR, le code de la PR ; pas le raisonnement de l'auteur.

Écrit. rien dans le dépôt : un commentaire sur la PR, dont la première ligne est exactement `Verdict: approve`, `Verdict: request_changes` ou `Verdict: block`, puis les défauts du plus grave au moins grave, avec ce qui les corrigerait.

Critères.

1. **rétrotraduction** : réécrire en mathématiques chaque énoncé Lean à partir du Lean seul, et comparer au texte (question, source, rédaction) ;
2. **lecture adversariale** : énoncé vide, quantificateurs, contenu caché dans les hypothèses, cas dégénérés (ensemble vide, zéro, division par zéro, soustraction dans $\mathbb{N}$, indices à partir de 0) ;
3. **règles d'`AGENTS.md`** : énoncés gelés intacts, variantes justifiées, définitions contrôlées, pas de définition évaluative, aucun théorème perdu, catalogue et question à jour, chemins humains intacts ;
4. **réutilisation** : rien de redéfini qui existe déjà ;
5. **citations** : chaque `[CLÉ, ...]` existe, passage lu ou marqué « to be checked », rubrique Attribution présente, rien d'annoncé comme nouveau avant l'audit.

Contrôles. le script n'accepte qu'un avis qui commence par `Verdict:` ; le relecteur tourne sur un autre modèle que l'auteur par défaut (Sonnet contre Opus).

7. Jardiner — rôle `garden` (modèle auteur)

Déclencheur. un tour sur dix.

Fait. une consolidation, la plus utile : une implication entre résultats du catalogue (cas particulier devenu corollaire, l'énoncé restant), une borne inférieure et une borne supérieure de même classe confrontées (si Lean prouve `False`, une définition est fausse : le signaler), un doublon avec les dépendances supprimé, le catalogue mis à jour.

Critères. englobement seulement à contenu quantitatif au moins aussi bon ; **aucun théorème perdu** (l'ancien énoncé doit rester prouvable) ; ne rien changer si rien d'utile.

Contrôles. CI et audit ; relecture ; seul un humain peut supprimer un énoncé.

3 Tes tâches et celles des automates

Qui	Quoi
Toi	écrire, relire et approuver les programmes ; décider des questions pendant la calibration ; fusionner les PR (avec le PDF résumé) ; auditer de temps en temps quelques questions acceptées et quelques fiches de lecture ; décider des suppressions d'énoncés ; protéger `main`.
Boucle	choisir la tâche (section 1) ; attendre le quota avec le calcul du worker Tau Ceti (`INCOGNITA_PACE`) ; fusionner automatiquement les catégories autorisées (`INCOGNITA_AUTOMERGE`, vide par défaut ; jamais `Statements/` ni les chemins humains).
Script d'agent	copie de travail isolée ; outils restreints ; refus des chemins interdits ; push et ouverture de la PR ; publication de l'avis du relecteur.
CI	compilation de la bibliothèque sans `sorry` ni avertissement, des énoncés gelés et des brouillons ; `lake exe audit` ; linters de Mathlib ; versions épinglées comme LMLExplore ; racines complètes ; interdiction d'importer `Exploration` ; PDF résumé de chaque PR.

4 Comparaison avec LMLExplore

4.1 Ce que fait le worker Tau Ceti

D'après les consignes de LMLExplore et ton guide du worker, chaque ronde fait *une* tâche, la plus utile disponible. Il y en a **trois**, pas deux :

1. **s'occuper de ses propres PR** : les rebaser, réparer la CI, répondre aux relecteurs (c'est la priorité) ;
2. **relire les PR des autres**, selon des grilles fixes : correctness, reuse, scope, attribution, API design, generality, placement, naming, documentation, proof quality. Chaque grille donne un verdict `approve`, `request_changes` ou `block` ;
3. **écrire une nouvelle PR** qui fait avancer une cible d'une feuille de route.

Autour : les humains écrivent et relisent les feuilles de route ; une PR est fusionnée automatiquement quand toutes les grilles approuvent et que la CI est verte ; le worker d'un mainteneur peut finir une PR inactive depuis un jour ; un script signale les doublons avec les dépendances.

4.2 Tableau comparatif

	LMLExplore	Incognita
Origine des énoncés	feuilles de route humaines, résultats publiés	programmes humains, questions proposées par des agents
Vérité de l'énoncé	supposée (vient de la littérature)	inconnue : preuve, réfutation ou variante justifiée
Validation de ce qu'on fait	revue humaine des feuilles de route	programme approuvé par toi, question validée par un agent (toi pendant la calibration)
Tâches des agents	3 : corriger ses PR, relire, écrire une PR	7 : proposer, valider une question, chercher, formaliser, bibliographe, relire, jardiner
Relecture	10 grilles, une par aspect	1 relecteur qui couvre 5 aspects (dont citations)
Indépendance	mêmes modèles pour écrire et relire	relecteur sur un autre modèle par défaut, ne voit pas le raisonnement
Énoncé gelé	non (fidélité jugée par la grille correctness)	oui, vérifié par `lake exe audit`
Résultats négatifs	pas de place prévue	réfutations et impasses consignées
Bibliographie	citations dans les docstrings, liens vers la littérature	bibliographie vérifiée, passages lus, audit d'attribution
Consolidation	doublons avec les dépendances	jardinage : englobement, aucun théorème perdu
Confiance	fusionné ou non	niveaux : fusionné, contrôlé, utilisé, établi
Fusion	automatique (toutes les grilles + CI)	par toi ; automatique par catégorie si tu l'actives
Quota	`-pace` du worker	le même calcul, via `tauceti status`
Résumé pour l'humain	description de la PR	description + PDF avec les énoncés Lean

Ce qui manque

— **« Corriger ses propres PR » n'existe pas encore comme tâche.** `AGENTS.md` la met en première priorité, mais aucun rôle ni la boucle ne l'implémentent : une PR qui reçoit `request_changes` ou dont la CI échoue reste bloquée jusqu'à ce que toi ou moi intervenions. C'est la première chose à ajouter (rôle `fix-pr` : rebaser, réparer la CI, traiter les défauts signalés, puis nouvelle relecture).

— **Un seul relecteur** contre dix grilles chez LMLExplore : moins coûteux, mais chaque aspect est regardé moins attentivement. On pourra découper la relecture en grilles si la calibration montre des oublis.

— **Double formalisation** des énoncés principaux et **niveaux de confiance** : écrits dans les règles, mais aucune tâche ne les déclenche encore.

5 Premier essai réel (13 novembre 2026)

Sur le serveur, une tâche de bibliographe puis une relecture, lancées à la main avec `scripts/agent.sh`.

— **Bibliographe** (Opus, quelques minutes). Il a constaté qu'aucun résultat n'était à auditer, puis a vérifié la première référence non vérifiée citée par un programme, `AG12` (Agrawal–Goyal, COLT 2012) : titre aligné sur l'éditeur, nom exact des actes, volume, pages, éditeurs, identifiant arXiv. Il a ouvert la PR #5 avec un résumé correct, et a signalé de lui-même

une ambiguïté : le programme RegretShape cite « Agrawal–Goyal » sans année, alors que les mêmes auteurs ont des articles de 2013 et 2017 plus proches du sujet (termes d'ordre 2).

— **Relecteur** (Sonnet, sans voir le raisonnement de l'auteur). Avis `Verdict: request_changes`, publié sur la PR, avec quatre défauts dont deux sérieux : le statut `checked` ne s'appuie sur aucune trace de la comparaison avec la source, et l'identification de la référence n'est qu'une inférence. Il a aussi dit honnêtement qu'il n'avait pas ouvert les pages sources lui-même.

— **Ce qui a fonctionné** : copie de travail, outils restreints et accès web, commit avec l'identité des agents, contrôle des chemins, push, PR, résumé, PDF en CI, avis du relecteur.

— **Bug trouvé** : le script tentait de supprimer la branche de relecture avant la copie de travail qui l'utilisait (sans conséquence, mais une branche restait sur le serveur) ; corrigé.

— **Ce que l'essai montre** : la PR #5 attend maintenant des corrections, et aucune tâche ne les fera, puisque « corriger ses propres PR » n'existe pas encore (section 4).